

PROBLEM ID: SIH26081

MINISTRY OF EARTH SCIENCES (MOES)

NCMRWF

THEME: DISASTER MANAGEMENT

ForeBlendCast: Multi-Model Weather Blend

Hybrid AI-NWP Multi-Model Forecast Blending & Calibrated Disaster Risk Early Warning System

REPOSITORY

foreblendcast

FRONTEND TECH

React 19, TS, Vite, Leaflet

BACKEND TECH

Python 3.11, xarray, zarr

CREDENTIALS NEEDED

None (Open & Public)

1. What the Project Is (Core Overview)

ForeBlendCast solves a critical national challenge posed by the **Ministry of Earth Sciences (MoES)** and **NCMRWF**: how to intelligently blend multiple weather forecasting systems—combining modern deep-learning AI weather models with traditional physical Numerical Weather Prediction (NWP) systems.

The Scientific Dilemma

Different models excel under different conditions:

- **ECMWF IFS HRES (NWP)**: High spatial fidelity; reliable for orographic rainfall.
- **ECMWF IFS ENS (Ensemble Mean)**: Smooth, reliable long-lead tracking.
- **Google DeepMind GraphCast (AI)**: Superior speed and mid-range synoptic tracking, but smooths local extremes.
- **Traditional Arithmetic Mean**: Fails catastrophically during monsoons because averaging smooths out the highest rain peaks, causing missed flood warnings!

The ForeBlendCast Solution

An end-to-end operational pipeline that achieves:

- **Canonical IMD 0.25° Alignment**: Standardizes all models to the official 03:00 UTC → 03:00 UTC (08:30–08:30 IST) rainfall accumulation cycle.
- **Context-Aware Adaptive Weights**: Weights change dynamically by lead day, terrain zone, season, and active/break monsoon regime.
- **Probability-Matched (PM) Blending**: Preserves heavy rainfall distributions so disaster thresholds remain intact.
- **District Risk Tiers**: Assigns Green, Yellow, Orange, and Red alerts to all 730+ Indian districts.

2. System Architecture & Data Contract

The project is engineered as a clean, decoupled system where backend data science and frontend user experience operate independently via a strict, file-based JSON/raster contract:

Architecture Workflow:

WeatherBench 2 (GCS) + ERA5 / IMD Reanalysis → **Ingestion & Alignment** (canonical/) → **Adaptive Weighting Engine** (weighting/) → **Probability-Matched Blending** (blending/) → **Calibrated Hazard Tiers** (hazards/) → results/*.json + results/rasters/*.png → frontend/public/data/ → **React Leaflet / Plotly Dashboard**

Contract File	Generated By	Purpose in Frontend Dashboard
results/ladder.json	experiments/run.py (Evaluation)	Renders the Evaluation Ladder (Climatology → Single Models → Blend → Oracle ceiling).
results/districts_L{1..10}.json	hazards/district.py	Drives the interactive India Map: district polygons, risk tiers (Yellow/Orange/Red), population affected, and model weights.
results/points/{city}.json	experiments/run.py	Ensemble spaghetti plots & confidence bands (q05–q95) for Delhi, Mumbai, Kolkata, Chennai, and Guwahati.
results/rasters/*.png	outputs/render_png.py	Georeferenced PNG overlays for rain intensity, exceedance probabilities, and model disagreement.
results/where_we_lose.json	evaluation/where_we_lose.py	Scientific integrity panel detailing exactly which regimes/districts the blend loses to single models.

3. Credentials, Keys & .env — What to Ask Your Friend

✓ No Private API Keys or Cloud Credentials Required!

You **do not need to ask your friend for any API keys, secrets, or cloud tokens** to run this project. All data sources used in the repository are completely public and open:

- **Google Cloud WeatherBench 2 (WB2):** Accessed with public anonymous access (`storage_options={"token": "anon"}` in `ingestion/wb2.py`). No Google Cloud account, IAM key, or service account JSON is needed.
- **Boundary & Demographic Data:** GeoJSON district boundaries download from open `geoBoundaries` (GitHub raw), and UN population rasters download from `WorldPop.org` .
- **Map Tiles:** Uses open Leaflet tiles with OpenStreetMap. No Mapbox tokens or Google Maps keys are required.
- **The `.env` file:** Listed in `.gitignore` purely as standard security hygiene. No environment variable files are used or required by either the backend or the frontend.

💡 What You SHOULD Ask Your Friend For Instead:

While no keys are needed, running the data pipeline from scratch downloads gigabytes of meteorological data and performs compute-heavy array operations. **Ask your friend for these pre-generated folders:**

1. **The `results/` folder:** If your friend has already run the experiment, get their generated `results/` folder. Copying it into your root directory will give you instant real outputs without waiting hours for downloads.
2. **The `data/static/` folder:** Contains pre-rendered district masks (`districts.geojson` , `districts_index.json` , `grid_static.nc`).

4. Commands to Run the Frontend

The frontend is fully built with **Vite, React 19, TypeScript, Tailwind CSS, Leaflet, and Plotly.js**. It already comes equipped with pre-bundled public data fixtures in `frontend/public/data/` , meaning **you can launch it immediately without running the backend!**

Step-by-Step Frontend Launch (PowerShell)

```
# 1. Open PowerShell and navigate to the frontend directory:
cd e:\foreblendcast\frontend

# 2. Install dependencies (Node.js & npm are already on your machine):
npm install

# 3. Start the Vite development server:
npm run dev
```

Once launched, open your web browser at: `http://localhost:5173` (or the URL displayed in your terminal).

Available Frontend Script	Command	What It Does
Dev Server	<code>npm run dev</code>	Starts the local Vite server with Hot Module Replacement (HMR).
Production Build	<code>npm run build</code>	Compiles TypeScript and outputs optimized production assets to <code>dist/</code> .
Demo Build Gate	<code>npm run build:demo</code>	Safety check: verifies that real data is present and rejects fixture data.
Linter	<code>npm run lint</code>	Runs Oxlint for high-speed code quality checks.

5. Commands to Run the Backend Science Pipeline

The backend is designed as a modular Python pipeline that pulls WeatherBench 2 inputs, computes weights and blends, categorizes district hazards, and generates output JSONs and PNG rasters.

A. Setup Environment

Option A (Conda / Mamba — Recommended for geospatial C-libraries):

```
# Create the environment with all geospatial C-libraries:
mamba env create -f environment.yml --force
conda activate sih26081
```

Option B (Standard Python Virtual Environment):

```
cd e:\foreblendcast
python -m venv .venv
.\venv\Scripts\activate
pip install numpy pandas xarray dask zarr gcsfs netcdf4 h5netcdf scipy scikit-learn lightgbm
pip install properscoring xskillscore
```

B. Running Pipeline Stages

```
# Step 1: Ingest static layers (district polygons, land mask, population)
python -m ingestion.static

# Step 2: Download raw forecast chunks from WeatherBench2 (GCS)
python -m ingestion.download

# Step 3: Run the full experiment pipeline for a forecast cycle
python -m experiments.run --cycle 2022-06-14

# Step 4: Render georeferenced raster PNGs
python -c "from outputs.render_png import write_bounds; write_bounds()"

# Step 5: Sync results into frontend public data folder (Windows PowerShell)
robocopy results frontend\public\data /MIR /NFL /NDL /NJH /NJS

# Step 6: Run test suite to verify accumulation windows & zero data leakage
python -m pytest tests/ -v
```

6. Repository Structure Reference

Directory	Role & Key Modules
<code>canonical/</code>	Defines India 0.25° grid bounds, accumulation window (03Z–03Z), unit converters, and quality gates.
<code>ingestion/</code>	WeatherBench 2 adapters (<code>wb2.py</code> , <code>ecmwf.py</code> , <code>graphcast.py</code>) and static data fetcher (<code>static.py</code>).
<code>weighting/</code>	Error tracking, inverse-error weights, empirical Bayes shrinkage, and human forecaster override handling.
<code>blending/</code>	Deterministic weighted mean and Probability-Matched (PM) blending algorithm (<code>probability_matched.py</code>).
<code>hazards/</code>	Precipitation exceedance probabilities, district aggregation, and model disagreement indexing.
<code>verification/</code>	Scientific verification: RMSE, MAE, Fractions Skill Score (FSS), Relative Economic Value (REV), and Reliability curves.
<code>frontend/</code>	Vite + React 19 interactive operational dashboard with Leaflet map, city spaghetti plots, and decision cards.
<code>tests/</code>	Guard tests preventing data leakage (<code>test_leakage.py</code>) and accumulation offset errors (<code>test_accumulation.py</code>).